

PatternShop — Capstone Project Brief

Contents

1	1. Overview	1
2	2. What you build	2
	2.1 2.1 Mandatory features	2
	2.2 2.2 Business rules	2
	2.3 2.3 Extension features	4
3	3. Pattern requirements	5
4	4. Spec first: your SPEC.md	5
	4.1 4.1 SPEC.md template	5
5	5. Architecture rules	6
6	6. What you are given	6
7	7. Testing expectations	7
8	8. Running the acceptance tests	7
9	9. Deliverables	7
10	10. Timeline (W9–W14)	8
11	11. Grading	8
12	12. Academic integrity and AI assistants	8

Course: Design Patterns in Java (Java 27), Fall 2026 · Runs W9–W14 · Your spec is due in W10 · Presentations in W14 · Grading: [rubric](#) · Türkçe: [spec.tr.md](#) · Engineering spec for instructors: [SPEC-capstone.md](#)

1 1. Overview

In the modules you met each pattern inside a few classes. In the capstone you combine them in one small, complete system: **PatternShop**, the order-processing core of an online shop. It runs on the command line, keeps all data in memory and has no UI and no database. What makes it interesting is the design:

- you write **your own SPEC.md first** (spec-driven development) and keep it up to date;
- you implement the mandatory features behind a **given API**, so a published **acceptance test suite** can tell you when a feature is done;
- you use and **justify at least ten design patterns**, plus modern Java 27 and one concurrency pattern;
- your code follows a **hexagonal architecture** that executable **ArchUnit rules** check on every build;
- you add **two extension features** of your own choice;
- you explain your design in a **report** and defend it in a **short presentation**.

There is no single right design. The acceptance tests check *behaviour*; the rubric rewards *well-justified design*. A pattern used where it solves nothing costs points (remember "patternitis" from m11).

2 2. What you build

2.1 2.1 Mandatory features

Id	Feature	In short
F1	Catalogue	Add products (physical or digital), find by SKU, search by category / maximum price / name, restock
F2	Cart	Open a cart for a customer, add items (same SKU merges), change quantity, remove, apply a coupon
F3	Undo / redo	Undo and redo cart edits, per cart, at most 20 steps back
F4	Pricing and promotions	Buy-X-get-Y-free, category percentage off, amount off over a threshold, coupons, shipping fee — applied in a fixed order
F5	Checkout with validation	A chain of validation rules that reports <i>all</i> problems in a fixed order
F6	Payment via a fake external API	Charge and refund through a third-party style API with an awkward interface (strings, status codes)
F7	Order lifecycle	PLACED → PAID at checkout, then SHIPPED → DELIVERED ; a paid order can be cancelled (refund + restock)
F8	Events and notifications	Domain events dispatched <i>after</i> the change is stored; customers notified; low-stock alerts
F9	Concurrent fulfilment	Ship all paid orders in parallel on virtual threads, with a limit on parallel orders
F10	Reports	Daily sales, top products, customer statement, inventory — as sealed types, rendered as text and CSV
F11	Command-line interface	A line-based CLI over all of the above

2.2 2.2 Business rules

These rules are what the acceptance tests check. Exact output texts (CLI, reports) are in the [test resources of the starter](#) and in [SPEC-capstone.md](#), "Output formats". If you find a contradiction between this brief and a test, report it — do not change the test.

Money and ids. All prices are Turkish lira with VAT included, stored as whole `kuruş` (Money). Printed as `987.91`. Ids are generated per shop instance: carts `cart-1`, `cart-2`, ...; orders `order-1`, `order-2`, ... (an order number is used only by an order that was actually placed). The clock is injected — never call `Instant.now()` directly. Invalid input (a malformed SKU, a blank name, a non-positive quantity, an unknown product added to a cart, ...) is rejected with an `IllegalArgumentException`; an unknown cart, or a restock of an unknown product, with a `NoSuchElementException`; any edit of a closed cart with an `IllegalStateException`. The exact messages are in the Javadoc of the GIVEN use cases.

Catalogue (F1). A SKU looks like `B0K-001` (three capitals, dash, three digits). Name must not be blank, price must be positive, a physical product's stock must be ≥ 0 , a digital product has unlimited stock (its stock is given and shown as 0). A duplicate SKU is rejected. Search results are sorted by SKU. Only physical products can be restocked, by a positive quantity.

Cart (F2). Adding a SKU that is already in the cart increases its quantity; the line keeps its first position. Quantities must be positive; changing a quantity to 0 removes the line. Unknown products are rejected and the cart stays unchanged. Stock is **not** checked while shopping, only at checkout. A cart holds at most one coupon; it is validated when applied (unknown or expired coupons are rejected; a coupon is valid up

to and including its `validUntil` day in the clock's time zone); applying another valid coupon replaces the first. After a successful checkout the cart is closed and every further edit is rejected.

Undo / redo (F3). Every successful edit (add, change quantity, remove, coupon) can be undone; undo restores the cart exactly, including line positions. Redo re-applies an undone edit; a new edit clears the redo history. History is per cart and keeps the last 20 edits. Failed edits are not recorded. Undo / redo with nothing to undo / redo report `false` and change nothing.

Pricing (F4) — always in this order:

1. Line total = quantity × unit price; *subtotal* = sum of line totals.
2. **Buy X get Y free** on a SKU: for every complete group of X + Y units, Y units are free.
3. **Category percentage off:** for every line in the category, the percentage of what is left of that line after step 2, rounded half-up to the kuruş, per line.
4. **Amount off over a threshold:** if subtotal minus the discounts of steps 2–3 is at least the threshold, subtract the amount. If several qualify, only the one with the highest threshold applies.
5. **Coupon:** its percentage of what is left after step 4, rounded half-up once. An expired coupon gives no discount.
6. The merchandise total never goes below 0.00: a discount is capped at what is left, and discounts of 0.00 are not listed (total = subtotal – discounts + shipping).
7. **Shipping:** 49.90 if the cart contains a physical product and the merchandise total is below 500.00; otherwise 0.00. Total = merchandise total + shipping.

Discount labels: `buy 2 get 1 free: TOY-001`, `10% off BOOKS`, `100.00 off over 1000.00`, coupon `AUTUMN5 5%`.

Worked example (sample catalogue and promotions shipped with the starter):

Step	Detail	Amount
Cart	BOK-001 ×2 (250.00), BOK-002 ×1 (400.00), TOY-001 ×3 (120.00), DIG-001 ×1 (99.90, digital, BOOKS)	
1 Subtotal	500.00 + 400.00 + 360.00 + 99.90	1359.90
2 Buy 2 get 1 free: TOY-001	one free unit	−120.00
3 10% off BOOKS	50.00 + 40.00 + 9.99	−99.99
4 100.00 off over 1000.00	1139.91 ≥ 1000.00	−100.00
5 Coupon AUTUMN5 5%	5% of 1039.91 = 51.9955 → 52.00	−52.00
7 Shipping	987.91 ≥ 500.00	0.00
Total		987.91

Checkout (F5, F6). Validation runs before anything is charged. If the cart is empty, the only reason is `empty cart`. Otherwise every rule runs and all reasons are reported in this order: `missing address` (only if the cart has a physical product; an address is complete when no field is blank) → `quantity limit exceeded: <SKU>` (more than 10 units of one SKU; in cart order) → `insufficient stock: <SKU>` (physical products; in cart order) → `expired coupon: <CODE>` → `missing card token`. A valid cart is charged **exactly once** through the external payment API: amount as text ("`987.91`"), currency "`TRY`", the merchant id from the settings and the cart id as idempotency key. Status 200 → the order is placed; 402 → `payment declined`; any 5xx → `payment unavailable`. A rejected checkout creates no order, changes no stock, keeps the cart open, publishes no event and sends no notification. A placed order reserves stock (physical products), closes the cart and records `PLACED`

and PAID with the clock's time and the provider's reference. A total of 0.00 is placed without calling the payment API (reference FREE).

Order lifecycle (F7). Allowed: PAID → SHIPPED (fulfilment only), SHIPPED → DELIVERED, PAID → CANCELLED. Cancelling needs a non-blank reason (otherwise missing reason), refunds the payment through the external API and restocks physical products; if the refund fails the result is refund failed and nothing changes. Anything else is refused with cannot cancel SHIPPED order, cannot deliver PAID order, ...; an unknown id with unknown order: <id>. Business outcomes are results (sealed types), not exceptions.

Events and notifications (F8). Events (OrderPlaced, OrderPaid, OrderShipped, OrderDelivered, OrderCancelled, StockLow) are dispatched only **after** the change is stored, in the order they were raised; a subscriber therefore sees the new state. Subscribers receive only their event type (a subscriber of ShopEvent receives all). A subscriber that throws is reported to the environment's error sink; the other subscribers still run and the operation still succeeds. Customers (recipient: the customer id) get a notification with the subject Order order-1 confirmed (on payment), Order order-1 shipped (with the tracking code) and Order order-1 cancelled (with the reason). When a product's stock drops from at least 5 to below 5, StockLow is published and ops is notified — once per crossing.

Fulfilment (F9). One call ships all orders that are PAID at that moment. Per order, in sequence: pick each physical line, pack, ship to the postal code (blocking calls to the warehouse API). Different orders run in parallel on **virtual threads**, never more than maxParallelOrders (default 4) at once. A failing order stays PAID and is reported with the exception message; the others still ship. A digital-only order ships without the warehouse with tracking code DIGITAL. Results — and the OrderShipped events, dispatched on the calling thread after the run — are in order-number order (order-2 before order-10). The call returns only when all work is finished.

Reports (F10). Daily sales (every day of the range, also days without orders; cancelled orders excluded), top products (by units, then SKU; list prices; cancelled orders excluded), a customer statement (all orders in placement order; total spent excludes cancelled orders), inventory (physical products by SKU, low when stock < 5). Requests and reports are sealed types; each report renders as text or CSV.

CLI (F11). help lists the commands. Every use case is reachable from the CLI; Main --demo starts with the sample catalogue and promotions. Unknown commands print ERROR unknown command: <word>, wrong arguments print the command's USAGE line.

2.3 2.3 Extension features

Choose **two** (pairs: three) and specify them in your SPEC.md with your own acceptance criteria and tests. Each one must add a design decision you can justify — not just code.

Id	Extension	Patterns it invites
E1	Product bundles with a bundle price	Composite
E2	Gift wrap and express handling options on lines	Decorator
E3	Promotion eligibility rules such as country = TR and spent >= 1000	Interpreter
E4	Catalogue import from CSV or JSON lines	Template Method, Adapter
E5	File-backed repositories that survive a restart (swap adapters in the composition root only)	Repository, Adapter
E6	Wish lists with price-drop alerts	Observer, Mediator
E7	Order export plug-ins discovered with ServiceLoader	Factory, plug-in

Id	Extension	Patterns it invites
E8	Payment resilience: retry and circuit breaker around the payment port	Decorator, Proxy
E9	Returns: DELIVERED → RETURN_REQUESTED → REFUNDED	State
E10	Fulfilment with Structured Concurrency (preview , optional) — isolated as in m10, default path stays on final APIs, all given tests must pass without --enable-preview	Structured Concurrency
E11	Your own idea, approved at the W10 spec review	—

3 3. Pattern requirements

- **At least 10 distinct patterns** from the creational, structural, behavioural and concurrency families, with at least **2 creational**, **2 structural**, **3 behavioural** and **1 concurrency** pattern. Immutable Object does not count as your concurrency pattern.
- Architectural patterns (Dependency Injection, Repository, Specification, Ports & Adapters, domain events) are required by the architecture anyway and **do not count** towards the ten.
- **Modern Java**: at least one **sealed** hierarchy of records that your code handles with an exhaustive **switch** using record patterns (no **default**). Records for values, lambdas for single-method strategies.
- **Concurrency**: fulfilment uses virtual threads with a bounded number of parallel orders (thread-per-task, Producer–Consumer, Guarded Suspension... — your choice, justified).
- Mark every participant type with the given annotation `@PatternRole` — the acceptance suite counts patterns from it, and the graders use it to find your code:

```
// snippet
@PatternRole(value = DesignPattern.STRATEGY, role = "concrete strategy")
record CategoryPercentOffRule(Category category, int percent) implements PromotionRule
{ ... }
```

- Every pattern appears in the **pattern-justification table** of your `SPEC.md` and report (template in the [rubric](#)).

4 4. Spec first: your SPEC.md

Before you write production code, write `capstone/starter/SPEC.md`. It is graded at W10 (rubric C1) and again at the end, together with its change log. Use the template below; keep it short and concrete — acceptance criteria must be checkable.

4.1 4.1 SPEC.md template

```
# PatternShop – <your name(s)>

> Status: draft | approved (W10) | final (W14) · Change log at the end

## 1. Objective
One paragraph: what the system does and for whom. Two or three user stories.

## 2. Scope
### 2.1 Mandatory features
| Id | Feature | Acceptance suite | My notes / interpretation |
### 2.2 Extension features
For each: description, user story, acceptance criteria (Given / When / Then), test class
```

```

names.
### 2.3 Out of scope

## 3. Domain model
Mermaid class diagram of the core types (aggregates, values, sealed hierarchies).

## 4. Architecture
Packages (domain, application, adapter.in, adapter.out, config), inbound and outbound
ports,
composition root. One Mermaid diagram of the hexagon.

## 5. Pattern plan
| # | Pattern (category) | Problem it solves here | Participants | Alternative
considered | Test |

## 6. Concurrency design
What runs in parallel, the limit, how failures and results are collected, why it is
thread-safe.

## 7. Testing strategy
Given acceptance tests, own unit tests (which test doubles), extension acceptance tests.

## 8. Boundaries and assumptions
Always / Ask first / Never. Assumptions you made where this brief is silent.

## 9. Milestones
Your plan for W10–W13 (which features and patterns per week).

## 10. Open questions and change log
| Date | Change | Why |

```

5 5. Architecture rules

Your code lives under `io.github.aliturgutbozkurt.patterns.capstone.shop` in the packages `domain`, `application`, `adapter.in.*`, `adapter.out.*` and `config` (Ports & Adapters, m11). The starter ships seven ArchUnit rules that run in every build and are already green on the skeleton. Keep them green — they are part of the grade and a red rule fails the implementation part:

1. The domain depends only on the JDK (`java.lang`, `java.util`, `java.math`, `java.time`), on itself and on the given value types, events and pattern annotations.
2. The application layer does not depend on adapters, on `config`, or on the external systems' APIs.
3. External systems (payment, warehouse, notifications) are reached only from outbound adapters and `config`; the simulators only from `config`.
4. Adapters do not depend on each other.
5. Adapters are wired only in `config` (the composition root); nothing depends on `config`.
6. No cycles between packages.
7. Production code depends only on the JDK and the course packages; no `System.exit`; no mutable static fields.

You may add your own rules; you may not delete or weaken the given ones.

6 6. What you are given

In `capstone/starter`:

- **GIVEN API** (`...capstone.api`, do not modify): value records (`Sku`, `Money`, `ids`, `Address`), one inbound port per feature (`CatalogueUseCase`, `CartUseCase`, `PricingUseCase`, `CheckoutUseCase`, `OrderUseCase`, `ShopEvents`, `FulfilmentUseCase`, `ReportUseCase`, `CommandLine`) gathered in `PatternShop`; the sealed request and result types; the external systems `ExternalPaymentApi`, `WarehouseApi`, `NotificationGateway`; deterministic simulators and `DemoData` for the CLI; the `@PatternRole` annotation.
- **Skeleton** (`...capstone.shop`): `config.ShopCompositionRoot` implements `PatternShopFactory` — the single place where the acceptance tests create your shop — and `config.Main`, plus empty packages with notes.
- **Tests**: 83 acceptance tests in 11 suites (`*ExerciseTest`, tag `exercise`), the 7 architecture rules (`ShopArchitectureTest`), test fakes (clock, payment sandbox, scripted warehouse, recording notifications) and the expected CLI transcript and report texts.
- **Templates**: `SPEC.md` and `REPORT.md`.

7 7. Testing expectations

- All **83 given acceptance tests** and all **7 architecture rules** pass. Never modify, delete, disable or weaken them, and never modify the GIVEN API (graders diff against the starter).
- **Your own unit tests** for the domain and application layers: every counted pattern has at least one test that shows its behaviour (e.g. a pricing step in isolation, a rejected state transition, a command and its inverse). Use hand-written test doubles (m11), no mocking library.
- **Your own acceptance tests** for each extension feature, traced to the acceptance criteria in your `SPEC.md`.
- Tests are deterministic: injected clock, no `sleep` to wait for threads (use latches or barriers as in m10).
- Coverage goal: $\geq 80\%$ line coverage of `domain` and `application` (`-Pcoverage`, JaCoCo).

8 8. Running the acceptance tests

Run every command from the repository root on JDK 27:

```
export JAVA_HOME=$(/usr/libexec/java_home -v 27)           # macOS; on Linux/Windows
point JAVA_HOME at JDK 27

./mvnw -q -pl capstone/starter verify                       # compile + your tests +
architecture rules (must stay green)
./mvnw -q -pl capstone/starter test -Pexercises             # the 83 acceptance tests
(red at the start)
./mvnw -q -pl capstone/starter test -Pexercises -Dtest='CartExerciseTest' -
Dsuresfire.failIfNoSpecifiedTests=false
./mvnw -q -pl capstone/starter verify -Pcoverage           # coverage report in
capstone/starter/target/site/jacoco

./mvnw -q -pl capstone/starter package -DskipTests
java -cp capstone/starter/target/classes
io.github.aliturgutbozkurt.patterns.capstone.shop.config.Main --demo
```

Work feature by feature: make one suite green, commit, move on. The order in §10 follows the module timeline.

9 9. Deliverables

All in your fork of the course repository, under `capstone/starter/`, on a tag `capstone-final`:

1. **Code** — `src/main` and `src/test`; `./mvnw -q -pl capstone/starter verify` and the `-Pexercises` run green.

2. **SPEC.md** — your specification (template §4.1) with the final change log.
3. **Tests** — own unit tests and extension acceptance tests (§7).
4. **REPORT.md** — 6–10 pages: design overview with the hexagon diagram, the pattern-justification table, the modern-Java and concurrency decisions, what you would change, and an AI-usage statement. Written in English **or** Turkish, with a one-page summary in the other language (terms per docs/glossary.md).
5. **Presentation** — 10 minutes + 5 minutes of questions in W14: a live CLI demo, three patterns in depth, one trade-off you made. Slides as presentation.pdf.

10 10. Timeline (W9–W14)

Week	Course module	Capstone work	Milestone
W9	m07 Observer, Chain, ...	Read the brief, fork, run the starter, read m11 §Ports and Adapters ahead; draft SPEC.md; choose extensions	M0 starter builds; verify green
W10	m08 State, sealed types	Finish SPEC.md; catalogue, cart, undo	M1 — SPEC.md due (graded, feedback within one week)
W11	m09 data-oriented	Pricing (Strategy + Decorator), order lifecycle (State), validation (Chain)	M2 Catalogue, Cart, Undo, Pricing suites green
W12	m10 concurrency	Checkout facade, payment adapter, events and notifications	M3 Checkout, Lifecycle, Events suites green
W13	m11 architecture	Concurrent fulfilment, reports, CLI, extensions, own tests	M4 all 83 tests + 7 rules green (feature freeze)
W14	Presentations	Report, slides, tag capstone-final before your slot	M5 final submission, presentation and defence

Commit at least once a week from W9 to W13 — the history is part of the evidence that the work is yours (rubric C8). SPEC.md may change after W10; record each change in the change log with its reason.

11 11. Grading

100 points, detailed in the rubric: specification and design 25 (C1–C2), implementation and tests 50 (C3–C8), report and defence 25 (C9–C10). In the syllabus the capstone is 40 % of the course grade (spec 10 % · implementation & tests 20 % · report & defence 10 %).

Automatic fail of the implementation part (C3–C8 = 0): the build or your own tests fail on JDK 27; any given acceptance test is red; any architecture rule is red; or a given test, rule or GIVEN API type was modified, deleted, disabled or weakened. The rubric lists these gates in full.

12 12. Academic integrity and AI assistants

- The capstone is **individual** work unless your instructor approved a pair in W9.
- You may reuse code from the course **modules** (examples and your own assignment solutions); mark it with a comment such as `// adapted from modules/m07-.../EventBus.java`. Unmarked reuse is plagiarism.

- The repository also contains a **reference solution** (capstone/reference). Copying from it, or from another student, is an integrity violation: the implementation part is graded 0 and the case follows the institution's procedure. Submissions are compared with the reference and with each other.
- AI assistants are allowed under the syllabus rules: disclose what you used them for in `REPORT.md`, and be ready to explain every line — the defence (rubric C10) asks about your own code. The `SPEC.md`, the pattern choices and their justification must be yours.